

Types construits : p-uplets, tableaux, dictionnaires

Ce que je vais savoir faire

- C1 — Je sais utiliser des p-uplets et écrire une fonction qui renvoie plusieurs valeurs.
- C2 — Je sais lire, modifier et construire des tableaux, y compris en compréhension.
- C3 — Je sais représenter et parcourir une grille avec un tableau de tableaux.
- C4 — Je sais créer, modifier et parcourir un dictionnaire.
- C5 — Je sais prévoir l'effet d'une affectation sur un objet mutable.

CARTE DU CHAPITRE

Prerequis : variables, boucles, fonctions, chaînes. Ce chapitre mène au mini-projet : classement d'un tournoi de e-sport.

Temps estimés : ♦ 12h ♦♦ 10h ♦♦♦ 9h

1.1 Les p-uplets (tuples)

DEFINITION D1

Un **p-uplet** (ou *tuple*) est une sequence ordonnee de p elements, eventuellement de types differents, separees par des virgules et entourees de parentheses. Un p-uplet est **non mutable** : une fois cree, on ne peut ni ajouter, ni retirer, ni modifier ses elements.

| Un p-uplet est une collection ordonnee et non modifiable de valeurs. En Python, on utilise le mot anglais *tuple*.

Exemple. Le tuple `station = ("Tunis", 36.8, 10.2)` contient trois elements.

```
1 station = ("Tunis-Carthage", 36.8, 10.2)
2 nom = station[0]           # "Tunis-Carthage"
3 latitude = station[1]      # 36.8
```

Contre-exemple. Toute tentative de modification provoque une erreur `TypeError`.

THÉORÈME Deballage

Une fonction peut renvoyer un tuple pour communiquer *plusieurs resultats* a la fois. On parle de **deballage** (*unpacking*) lorsqu'on affecte chaque element du tuple a une variable distincte : $x, y = f(a, b)$.

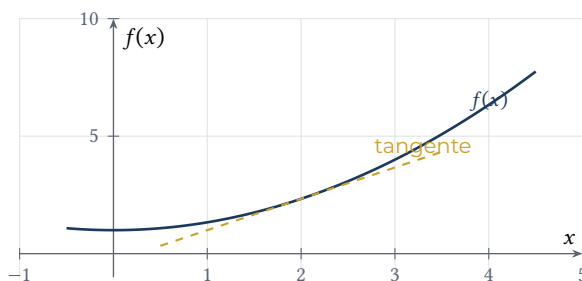
| Le deballage est tres utilise en Python. On ecrit `x, y = milieu(a, b)` plutot que `r = milieu(a, b)` puis `x = r[0]`.

ERREUR FRÉQUENTE

Modifier un tuple comme une liste (EF1). Un tuple est immuable par conception.

~~Copie fautive : `station[1] = 37.0`~~

Correction : creer un nouveau tuple ou utiliser une liste.



POUR ALLER PLUS LOIN

Un tuple de longueur 1 s'ecrit `(42,)` avec une virgule obligatoire. Sans virgule, `(42)` est simplement l'entier 42 entre parentheses. Le tuple vide s'ecrit `()`.

MÉTHODE M1 — Choisir entre tuple et liste

Quand l'utiliser. Tu hesites entre un tuple et une liste pour stocker tes donnees.

La méthode pas à pas.

1. Les donnees doivent-elles rester fixes apres creation? → **tuple**.
2. Faut-il ajouter, supprimer ou modifier des elements? → **liste**.
3. Les elements sont-ils de types differents (nom + age)? → **tuple**.

4. S'agit-il d'une collection homogene (scores)? → **liste**.

Exemple rédigé. Coordonnees GPS : ("Tunis", 36.8, 10.2) → tuple. Relevés horaires : [18, 20, 19] → liste.

Les pièges.

- Ne pas oublier la virgule pour un tuple d'un seul element.

Vérifier son résultat. Si tu as choisi un tuple, vérifie que tu ne tentes jamais de le modifier.

S'entraîner. Exercices 2, 4, 8.

Exercices

SPEC-EX-001

⌚ 5 min

Exercice 1

On donne station = ("Tunis", 36.8, 10.2).

1. Donner le type et le nombre d'elements.
2. Donner station[0] et station[2].
3. Peut-on écrire station[1] = 37.0 ?

SPEC-EX-002

⌚ 12 min

Exercice 2

Écrire une fonction au-dessus(scores, seuil) renvoyant le tableau des scores strictement supérieurs.

Écrire une fonction moyenne(scores) renvoyant la moyenne. Lever ValueError si le tableau est vide.

SPEC-EX-003

⌚ 20 min

Exercice 3

Écrire historique_scores(equipes, resultats) qui met à jour les scores **sans modifier** l'original. Justifier le choix de copie et analyser la complexité.

SPEC-EX-001

Corrigé

1. type(station) renvoie <class 'tuple'>. Le tuple contient 3 elements.
2. station[0] vaut "Tunis", station[2] vaut 10.2.
3. Non : un tuple est non mutable.

La question 3 teste la compréhension de la non-mutabilité (C1/C5).

Code de référence

CODE DE RÉFÉRENCE — Fonction renvoyant un tuple

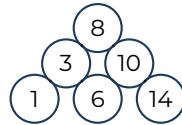
```
1 def milieu(a, b):
2     """Renvoie le milieu de deux points."""
3     mx = (a[0] + b[0]) / 2
4     my = (a[1] + b[1]) / 2
5     return (mx, my)
6
7 x, y = milieu((1, 2), (5, 8))
```

```
8 | print(x, y) # 3.0 5.0
```

```

>> milieu((0, 0), (6, 8))
(3.0, 4.0)
>> type(milieu((0, 0), (6, 8)))
<class 'tuple'>

```



MINI-PROJET — Classement d'un tournoi de e-sport

Un tournoi oppose 8 joueurs sur 5 manches. Chaque joueur est un dictionnaire avec les clefs "pseudo", "equipe" et "scores".

Jalon 1. Représenter les données : tableau de 8 dictionnaires.

Jalon 2. Calculer les totaux avec `total(joueur)`.

Jalon 3. Trier le classement par total décroissant (copie, pas alias).

Jalon 4. Afficher le classement formate.

Critères de validation (testables) : Jalon 1 : acces aux donnees OK ; Jalon 2 : totaux corrects ; Jalon 3 : original non modifie ; Jalon 4 : affichage complet.

QCM — Types construits

Pour chaque question, une seule reponse est correcte.

Q1. (C1) Que renvoie `type((3, 5))` ?

- A. `<class 'list'>`
- B. `<class 'tuple'>`
- C. `<class 'int'>`

Q2. (C5) `a = [1, 2]; b = a; b.append(3)`. Que vaut `a` ?

- A. `[1, 2]`
- B. `[1, 2, 3]`
- C. Une erreur

Diagnostics : voir la section Diagnostics du QCM en fin de chapitre.